

고급 컴퓨터 그래픽스(3207-001)

프로그래밍 Assignment #1 보고서

Draw Object(PiggyBank, Cube),

Spline Curve,

Animate



학과	컴퓨터공학부
학번	202112346
이름	정근녕

1. 구현 방법

3D 객체인 PiggyBank와 Cube를 로드하고, Hermite, Bezier, B-Spline으로 구현된 Spline Curve를 txt 형태로 3차원의 control Point를 입력받아 화면에 출력하도록 하였습니다. spline curve의 경로 좌표를 따라 Cube가 따라 움직일 수 있도록 하였습니다. 또한 키보드를 입력받아 spline을 변경할 수 있도록 구현하였습니다.

2. 구현 설명

2.1 **obj Load** : OBJ_Loader 라이브러리를 활용하여 '.obj' 파일(예: PiggyBank.obj, cube.obj)을 로드합니다.

2.2 **Object 클래스 구현(Object.h)** : 다양한 객체들의 데이터를 관리하고 화면에 그리기 편리하게 하기 위해서 'Object' 클래스를 구현하였습니다. 유니티 엔진과 같은 상용 엔진에서 객체들을 관리하는 것을 생각해 구현하였습니다.

2.2.1 **객체 관리** : 객체를 관리하기 위해서 여러 메소드를 정의하였는데 InitFromMesh() 함수를 통해 VAO, VBO, Texture 파일을 로드합니다. 텍스처는 stb_image 라이브러리를 사용하였습니다. Draw 함수를 구현해 함수 호출로 화면에 출력하도록 구현하였습니다.

2.2.2 **기타 메소드** 이러한 초기화 함수외에도 화면에 출력하는 기능과 animation을 위한 rotate 함수, spline 곡선을 따라 움직이도록 하기 위해서 좌표값으로 객체의 position을 변경하는 함수를 구현하였습니다.

2.3 **Spline 클래스 구현(spline.h)** : Object 클래스의 목적처럼, Spline을 더 쉽게 관리하기 위해서 작성하였습니다.

2.3.1 **객체 관리** : 생성자를 구현해 Point, Spline Shader를 로드하고, 셰이더를 위한 값들을 설정하도록 구현하였습니다. Object와 비슷하게 Draw 함수를 구현하였습니다.

2.3.2 **getPointOnSpline()** : 과제 조건의 객체의 애니메이션이 커브를 따르게 하도록 spline의 t값의 좌표를 반환하는 함수를 구현하였습니다.

2.3.3 **Spline 별 함수 구분(Hermite, Bezier, B-Spline)** : 각 내부 함수들은 키보드로 입력받은 CurveType에 따라 종류에 맞는 값을 반환하도록 분기처리로 로직을 나누어 구현하였습니다.

2.4 **셰이더** : 셰이더는 크게 3 종류로 구현하였습니다.

2.4.1 **객체(Object)를 위한 VertexShader.txt, FragmentShader.txt**

객체는 위한 셰이더는 그동안 많이 사용해온 Phong Shading을 지원하고 texture 파일이 있다면 base color로 texture의 색상값을 반영하도록 구현하였습니다.

2.4.2 **Spline_VertexShader.txt, Spline_FragmentShader.txt, Spline_GeometryShader.txt,**

셰이더는 기본적으로 spline_contol_point.txt에서 받은 control point 좌표를 받아 geometry shader로 넘기며, geometry shader에서 각 controlpoint 사이의 간격을 100으로 지정해 선분을 이루며, 선분의 좌표는 강의자료에 있는 Hermite 자료를 참고하여 작성하였습니다. Hermite를 구현하기 위해서는 기울기를 계산해야 하는데, 강의에서 나온 기울기 대신 기울기를 특정할 수 있는 점을 주는 알고리즘으로 구현하였습니다.

구체적인 알고리즘을 찾기위해 **AI를 사용**하였고, Catmull-rom Spline으로 구현하였습니다. 다른 알고리즘은 강의 자료에 나온 공식들을 사용해 구현하였으며, flat in int v_curveType; 으로 키보드로 입력받은 curve 종류 data를 가져와 하나의 셰이더 파일에서 3가지의 curve에 대한 계산을 가능하게 구현하였습니다.

2.4.3 **Control Point를 위한 Point_VertexShader.txt, Point_FragmentShader.txt**

직접 작성한 3차원 좌표가 적혀있는 spline_contol_point.txt 파일의 좌표를 받아 화면상

좌표를 계산하도록 구현하였습니다.

2.5 Scene(Scene.h, Main.c 내 Logic) 개념 : 상용 엔진처럼 만든 Object는 Scene 클래스에 넣어 관리하도록 하였습니다. Scene에 있는 객체를 loop를 통해 접근하여, Object, Spline 클래스에서 제공하는 메소드들로 동시에 제어하도록 구현하였습니다.

3. 구현 환경

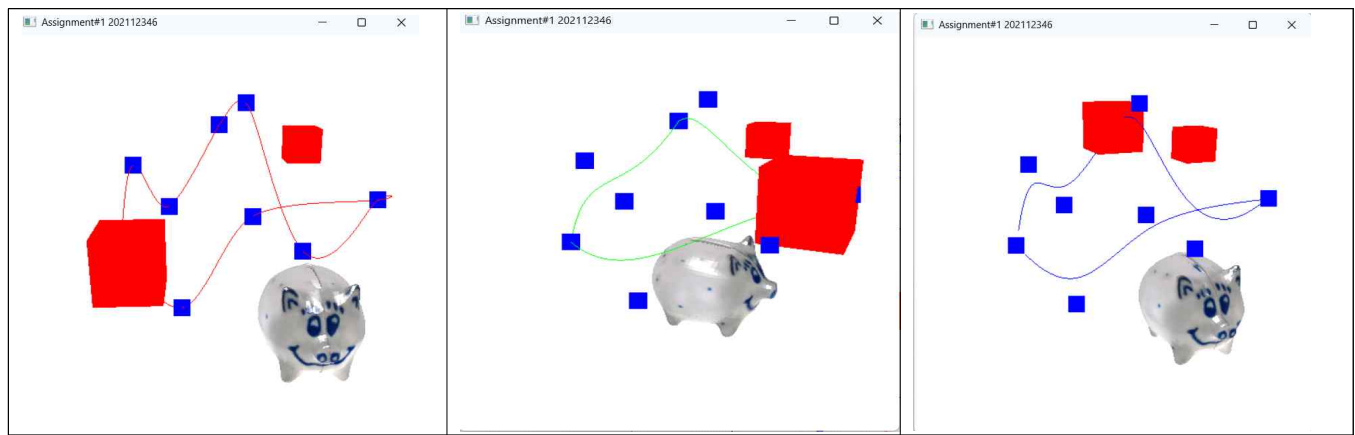
3.1 운영체제 : Window 11

3.2 컴파일러 : Visual Studio 2022

3.3 사용된 opengGL라이브러리 freeglut , glew, glm

3.4 사용된 openSource 라이브러리 : OBJ_Loader(<https://github.com/Bly7/OBJ-Loader>),
stb_image(<https://github.com/nothings/stb>)

4. 스크린샷



4.1 영상 :

https://drive.google.com/file/d/1KUcpErfZRjYFSe-eGI1mtgsDTaeLfcB_/view?usp=sharing

5. 기타 관련

5.1 Sample Cube Code, Homework#2 프로젝트 코드,

지난 Computer Graphics(차영운) 수업 및 OpenGL Bible을 학습하며 저장한 코드들을 참고하며 과제를 수행하였습니다. https://github.com/Kkackit02/OpenGL_API_Practice

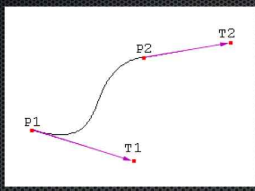
5.2 과제 구현 간 문법 교정, 단순 반복적 구현 및 해결이 어려운 버그를 디버깅하기 위해 Gemini CLI를 보조 도구로 사용하였습니다.

Console에 나타난 에러, 반복 작업에 대해서...

-> CLI가 수정 -> 분석 후 의도대로 수정

5.3 2.4.2 Hermite Spline Geometric Shader를 구현을 위해 AI를 사용하였습니다.

Hermite

- How to specify derivatives?
 - Control points
 
 - Long curves

$$x(0) = p1, x(1) = p2, x'(0) = t1 - p1, x'(1) = t2 - p2$$

$$x(u) = [2u^3 - 3u^2 + 1 \quad -2u^3 + 3u^2 \quad u^3 - 2u^2 + u \quad u^3 - u^2] \begin{bmatrix} x(0) = p1 \\ x(1) = p2 \\ x'(0) = t1 - p1 \\ x'(1) = t2 - p2 \end{bmatrix}$$

기울기는 어떻게 줄수 있을까?

↳ 기울기 대신 기울기를 특정할 수 있는 점을 주면

↳ T_1, T_2 T_1 에서 우편변개

$$x(u) = [B_0(u) \ B_1(u) \ B_2(u) \ B_3(u)] \begin{bmatrix} p1 \\ p2 \\ t1 - p1 \\ t2 - p2 \end{bmatrix}$$

(Hermite Geometry Shader 구현 중 구현 방식에 대해서 AI에게 질문하였습니다.)

-> 기울기 대신 기울기를 특정할 수 있는 점을 주는건 무슨 방식이야?

◆ 기울기(탄젠트) 벡터 대신, 그 기울기를 특정할 수 있는 '점'을 주는 방식은 Catmull-Rom 스플라인 또는 Cardinal스플라인을 의미하는 경우가 많습니다.

현재 셰이더 코드에서 사용하는 방식이 바로 그것입니다.

조금 더 자세히 설명해 드릴게요.

p_0, p_1, p_2, p_3 네 개의 제어점이 있다고 상상해 보세요. 우리는 p_1 에서 p_2 로 이어지는 곡선을 그리고 싶습니다.

* 시작점: p_1

* 끝점: p_2

이제 p_1 과 p_2 에서의 기울기(탄젠트)가 필요한데, 이 값을 직접 주는 대신 주변 점들을 이용해 계산하는 겁니다.

* ' p_1 '에서의 기울기: p_0 와 p_2 의 위치를 참고하여, p_0 에서 p_2 를 향하는 방향으로 결정됩니다. (수식: $T_1 = c * (p_2 - p_0)$)

* ' p_2 '에서의 기울기: p_1 과 p_3 의 위치를 참고하여, p_1 에서 p_3 를 향하는 방향으로 결정됩니다. (수식: $T_2 = c * (p_3 - p_1)$)

! Catmull-Rom Spline (https://upload.wikimedia.org/wikipedia/commons/thumb/4/4f/Catmull-Rom_spline_updated.svg/400px-Catmull-Rom_spline_updated.svg.png)

이 그림에서처럼, P_1 에서의 접선은 P_0 와 P_2 를 잇는 점선과 평행한 방향으로 만들어지고, P_2 에서의 접선은 P_1 과 P_3 를 잇는 점선과 평행한 방향으로 만들어집니다

결론적으로, "기울기 대신 점을 준다"는 것은 다음과 같습니다.

> Hermite 곡선의 한 종류로서, 탄젠트 벡터를 직접 입력하는 대신, 곡선 경로 주변의 추가적인 제어점을 이용하여 탄젠트를 암시적/자동으로 계산하는 방식입니다.

이 방식의 가장 큰 장점은 여러 점들을 부드럽게 통과하는 곡선을 쉽게 만들 수 있다는 점입니다. 애니메이션

경로, 카메라 패스, 3D 모델링 등에서 널리 사용됩니다.

5.4 큐브를 Spline을 따라 보낼 때, 큐브의 밝은 면은 원래 초기 생성위치에 남아 혼자 회전하고 있는 오류해결을 위해 SI를 사용하였습니다.

Cube가 Spline Line을 따라 이동하는데, 자꾸 Cube의 밝은 면만 남아서 가운데에 그대로 있어. 회전은 동시에 이뤄지는데

◆ 네, 흥미로운 현상이네요. 큐브가 회전은 하는데 이동은 하지 않고, 밝은 면만 중앙에 고정되어 있다는

말씀이시군요.

객체가 두 개 있는 것은 아닐 가능성이 높습니다. 그보다는 하나의 큐브에 적용되는 변환(Transformation) 행렬에

문제가 있을 확률이 큼니다. 증상을 들어보면, 다음과 같은 시나리오가 가장 유력합니다.

1. 회전은 적용되고 있다: 큐브가 회전하는 것으로 보아, '회전 행렬'은 올바르게 계산되어 셰이더로 전달되고

있습니다.

2. 이동은 적용되지 않고 있다: 큐브가 스플라인을 따라가지 않고 중앙에 머무는 것은, 스플라인의 위치 값으로

계산된 '이동 행렬'이 최종 변환에 누락되었기 때문입니다.

3. 밝은 면만 보인다: 큐브가 월드 공간의 원점(0, 0, 0)에서 움직이지 않기 때문에, 광원(Light)과의 상대적인

위치가 변하지 않아 계속 같은 면만 밝게 빛나는 것입니다.

결론적으로, 스플라인을 따라 계산된 '이동(Translation)' 정보가 최종 모델 행렬에 포함되지 않고, '회전(Rotation)' 정보만 적용된 모델 행렬이 셰이더로 넘어가는 것 같습니다.

확인해볼 지점:

큐브의 모델 행렬(Model Matrix)을 계산하고 셰이더에 전달하는 C++ 코드를 확인해 보세요.

AI는 해결을 못해서..

Scene에 들어가는 Object중 밝은 면이 index 4임을 확인해서 직접 묶어서 이동하게 해결했습니다.

-> if (getScene().size() > 1)

{

getScene()[1]->SetPosition(newPos);

getScene()[4]->SetPosition(newPos);

}

curveType이 제대로 전달되지 않아, 자꾸 곡선이 노란색으로 떠 프래그먼트 셰이더를 보면, curveType이 0(빨강), 1(초록), 2(파랑) 중 어느 것도 아닐 때 기본값으로 노란색을 표시하도록 했습니다. 노란색이 나왔다는 것은 지오메트리 셰이더에서 프래그먼트 셰이더로 curveType 변수가 제대로 전달되지 않고, 쓰레기 값(garbage value)이 들어갔다는 뜻입니다. 원인은 셰이더의 변수 보간(interpolation) 문제입니다. 정수(int) 타입의 변수는 셰이더 파이프라인을 거치면서 그래픽카드에 의해 자동으로 보간(중간값을 계산하는 과정)되려고 시도하는데, 이 과정에서 값이 깨져버립니다. 해결책: 이런 현상을 막기 위해 변수 앞에 flat 키워드를 붙여 "이 변수는 보간하지 말고, 그대로 전달하라"고 명시해야 합니다. 1. 지오메트리 셰이더: out int v_curveType; -> flat out int v_curveType; 으로 수정 2. 프래그먼트 셰이더: in int v_curveType; -> flat in int v_curveType; 으로 수정
--

셰이더에 있는 추가해둔 베지어, B-SPLINE 함수를 가져와서 GetPointOnSpline 함수에서 내가 선택한 베지어, b-spline 함수에 좌표를 불러오는 기능을 추가해줘 getPointOnSpline 함수를 지오메트리 셰이더처럼 수정하여, 현재 선택된 곡선 종류(m_currentCurveType)에 따라 Bezier와 B-Spline 공식을 올바르게 선택하여 계산하도록 변경하겠습니다. 이 작업을 위해 먼저 Spline.h 파일을 다시 읽어오겠습니다. ✓ ReadFile ACG_HomeWork\Spline.h ◆ 네, Spline.h 파일을 읽었습니다. 이제 getPointOnSpline 함수를 현재 선택된 곡선 종류에 맞는 계산을 하도록 수정하겠습니다. if/else 문을 통해 Bezier, B-Spline, Hermite 공식을 모두 처리하는 새로운 함수로 교체하겠습니다. 이렇게 하면 큐브가 항상 화면에 보이는 곡선과 동일한 경로로 움직이게 됩니다. Spline.h 파일의 getPointOnSpline 함수를 교체하겠습니다. ◆ getPointOnSpline 함수 수정이 완료되었습니다. 이제 이 함수는 현재 선택된 곡선 종류(Hermite, Bezier, B-Spline)를 올바르게 인지하고, 그에 맞는 공식을 사용하여 큐브의 위치를 정확하게 계산할 것입니다.
